SLIIT
Discover Your Future

SE2020 – Web Technologies and Mobile

Lecture 11

RESTful Web Services and Express JS

SLIIT
FACULTY OF COMPUTING

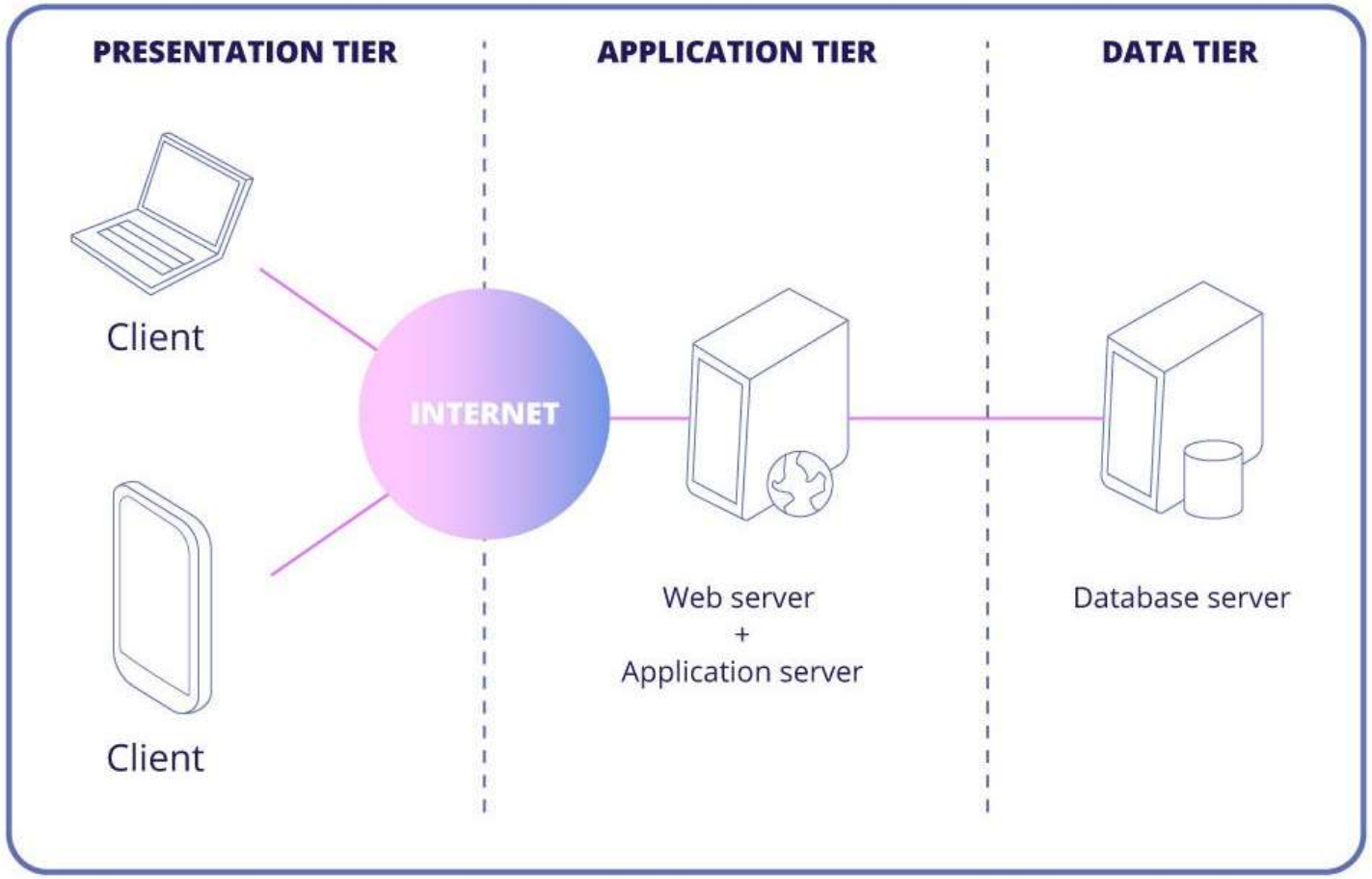
Agenda

- Introduction
- 3-Tier Architecture
- Application Layer
- MVC Architecture
- REST Service
- Express JS

Introduction

- As applications grow, we need to separate responsibilities – for easier maintenance, testing, and scalability.
- Modern applications follow architectural patterns that help achieve this structure.
- Today, we'll connect four key ideas:
  - 3-Tier Architecture – the overall system design
  - MVC – the internal organization (architectural) pattern
  - REST – the communication style
  - Express.js – the implementation framework

Three Tier Architecture



Why 3-Tier architecture for Web and Mobile apps?

- **Independent Infrastructure** – Each layer (Presentation, Application, and Data) runs on its own infrastructure, ensuring better performance and maintainability.
- **Parallel Development** – Different teams can develop each tier independently, improving efficiency and speeding up the development process.
- **Scalability** – You can scale up or down each layer separately, without affecting the other tiers, making resource management more flexible.
- **Enhanced Security** – Unlike two-tier architecture, this model provides better security by isolating data and logic from the frontend, reducing vulnerabilities.

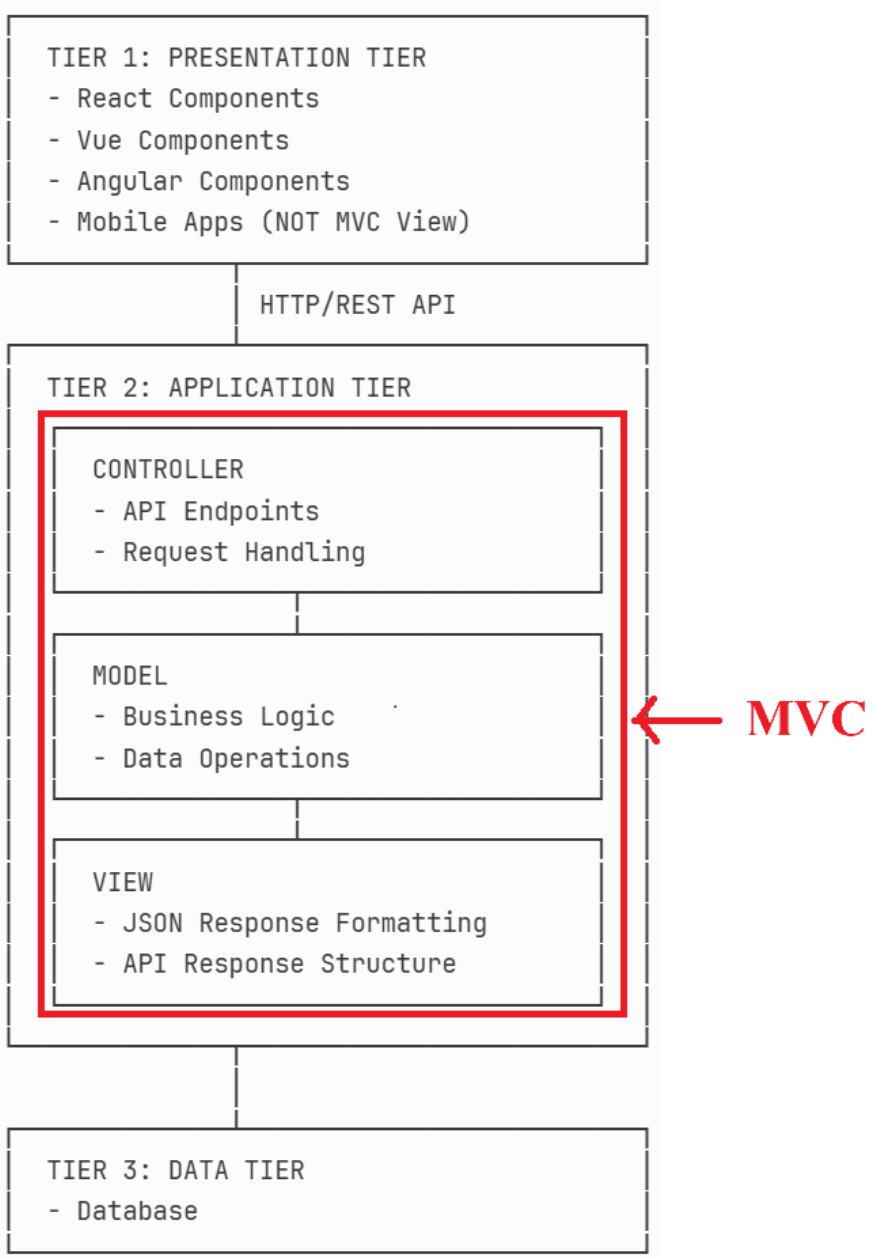
Presentation Layer

- The Presentation Layer, also called the **client-side or frontend**, is responsible for interacting with users.
- It is the **visible part** of the application where users input data and receive outputs.
- The main goal of this layer is to ensure a smooth **user experience** (UX) with responsive and accessible interfaces.
- Modern Frontend Frameworks and Libraries:
  - React.js, Angular, Vue.js.
- Legacy Frontend Technologies:
  - jQuery, AJAX (Asynchronous JavaScript and XML).

Application Layer

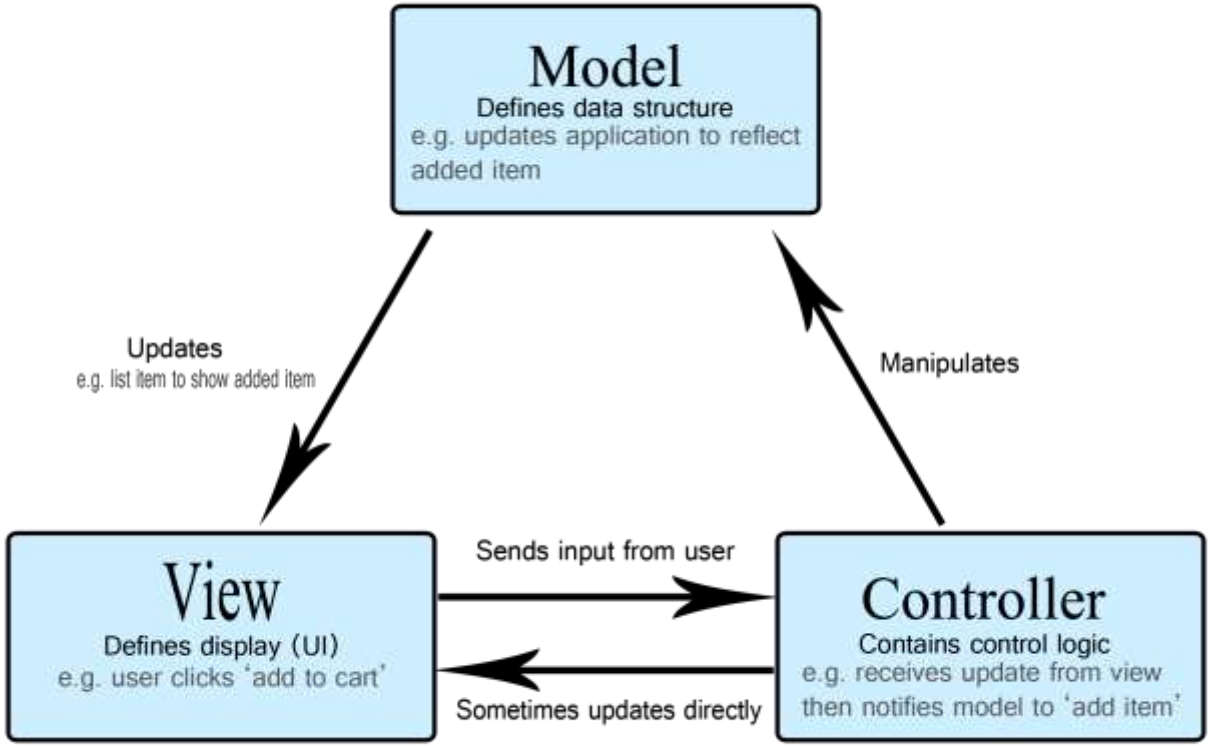
- The Application Layer, also called the server-side or backend, is responsible for processing business logic, handling requests, and managing data.
- It acts as an intermediary between the Presentation Layer (frontend) and the Database Layer.
- Technologies used to implement include but not limited to,
    - Java - Spring, Spring Boot
    - Node.js - Express.js, Koa
    - Python - Django, Flask
    - PHP - Laravel, Symfony
    - .NET - ASP.NET Core

MVC Architecture in 3-Tier



MVC Architecture

MVC (Model-View-Controller) is a software design pattern used to organize code in applications. It separates the application into three interconnected components, each with specific responsibilities.



MVC Architecture

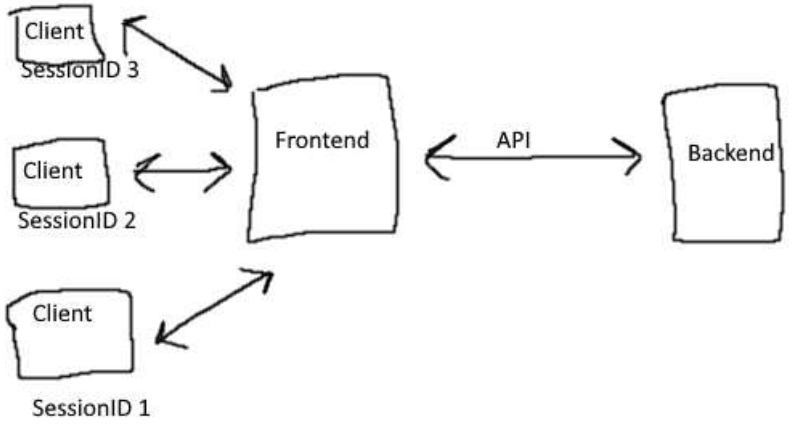
Model	Controller	View
Business logic	Send & Receive HTTP requests	Format data for presentation
Data validation	Validate request data	Structure API responses
Database operations	Call Model methods	Hide sensitive data
Business rules enforcement	Call View to format response	Create consistent response format

Presentation Layer <=> Application Layer

How does the Backend and the frontend communicate?

(REST API Communication Scenario)

- User Action – A user submits a form on the frontend (e.g., signing up).
- Frontend Request – The frontend sends a POST request to the backend with the form data.
- Backend Processing – The backend validates the data, updates the database, and processes business logic.
- Backend Response – The backend sends a success/failure response back to the frontend.
- Frontend Update – The frontend updates the UI dynamically based on the response (e.g., "Registration Successful!").



Application Layer : APIs

What is an API?

- API (**Application Programming Interface**) is a software interface that allows two or more applications to communicate with each other.
- It acts as a **bridge** between different software components, enabling seamless integration.
- APIs **abstract away complexities**, allowing developers to interact with services without knowing the internal implementation.
- It defines a **contract of services**, specifying how applications should request and exchange data

Application Layer : APIs

Benefits of API

- Standardized Communication
  - Uses standard protocols like HTTP/HTTPS, WebSockets, gRPC.
  - Responses are typically in JSON or XML format.
- Encapsulation & Abstraction
  - Hides complex implementation details, exposing only necessary functions and data..
  - Developers don’t need to understand database queries or backend logic.
- Reusability & Efficiency
  - APIs allow applications to reuse functionality, reducing redundant code.
  - Example: A payment gateway API (like Stripe) can be integrated into multiple applications without writing new payment logic.

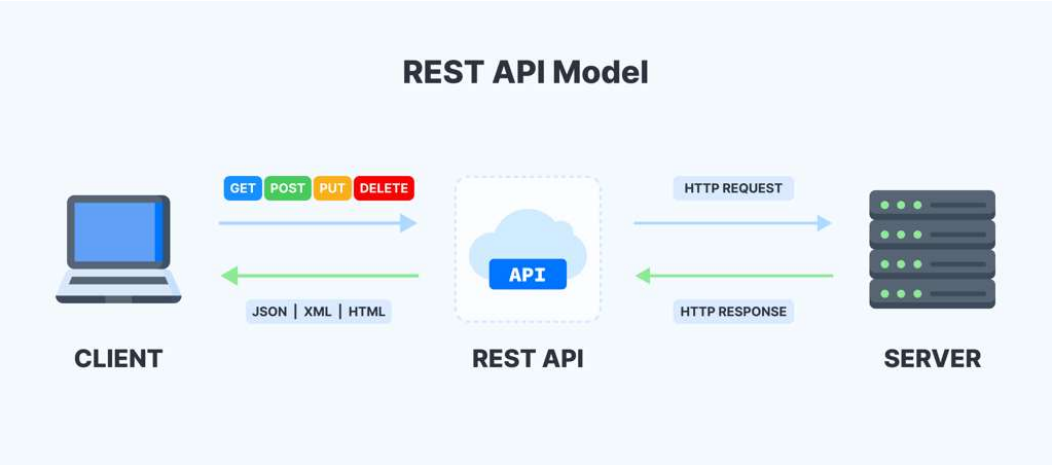
Application Layer : APIs

Different Types of APIs

- REST API (Representational State Transfer)
  - Uses HTTP methods (GET, POST, PUT, DELETE) for communication.
  - Stateless and follows resource-based architecture.
  - Responses are typically in JSON or XML.
  - Commonly Used For: Web and mobile applications, cloud services.
- SOAP API
- RPC (Remote Procedure Call)
- WebSockets API
- GraphQL API

REST

- REST (Representational State Transfer) is an architectural style for building web services.
  - A **web service** is a way for applications to exchange data and interact with each other, machine-to-machine communication over the web.
- It defines a set of constraints for how clients and servers should communicate over the web, using HTTP as the communication protocol.

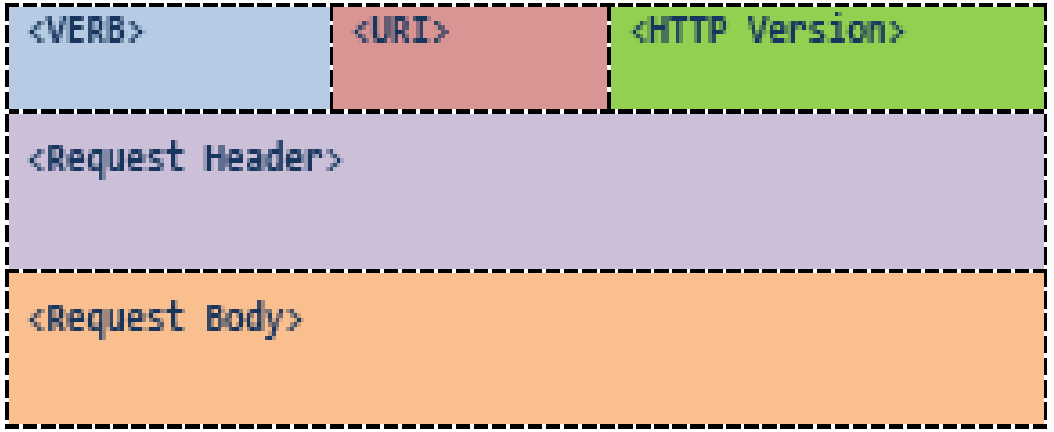


REST – Resources

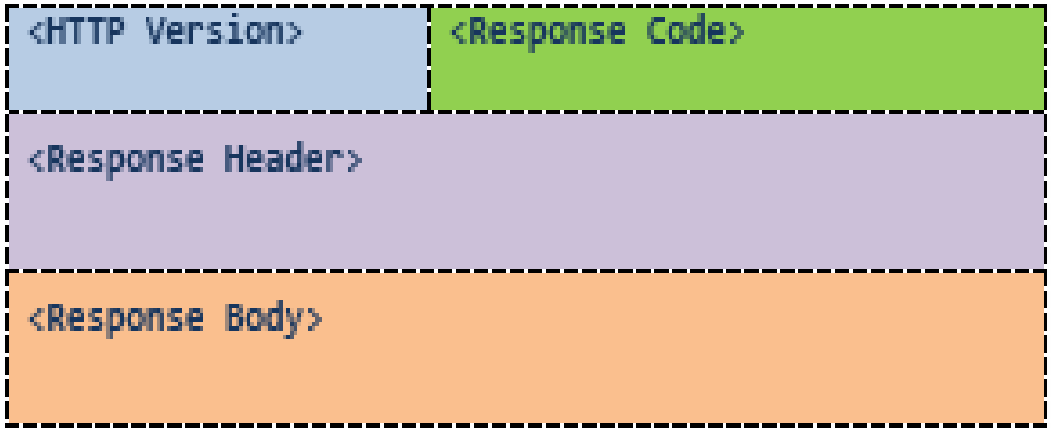
- In REST, everything is treated as a resource (e.g., user, product, order).
- Each resource is identified by a unique URI (e.g., /api/users/123).
- Clients use **HTTP methods** to perform operations on resources
  - GET → Retrieve data (GET /api/users)
  - POST → Create a new resource (POST /api/users)
  - PUT → Update an existing resource (PUT /api/users/123)
  - DELETE → Remove a resource (DELETE /api/users/123)

REST – HTTP Request and Response

Request Message



Response Message

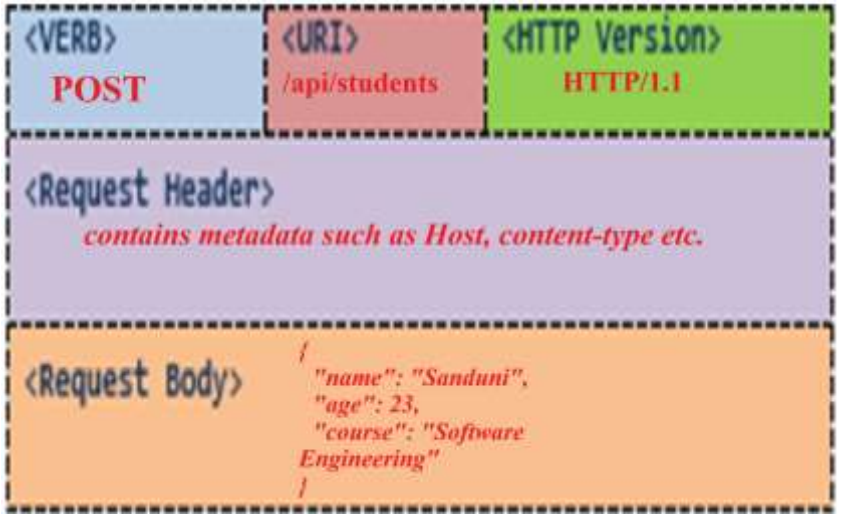


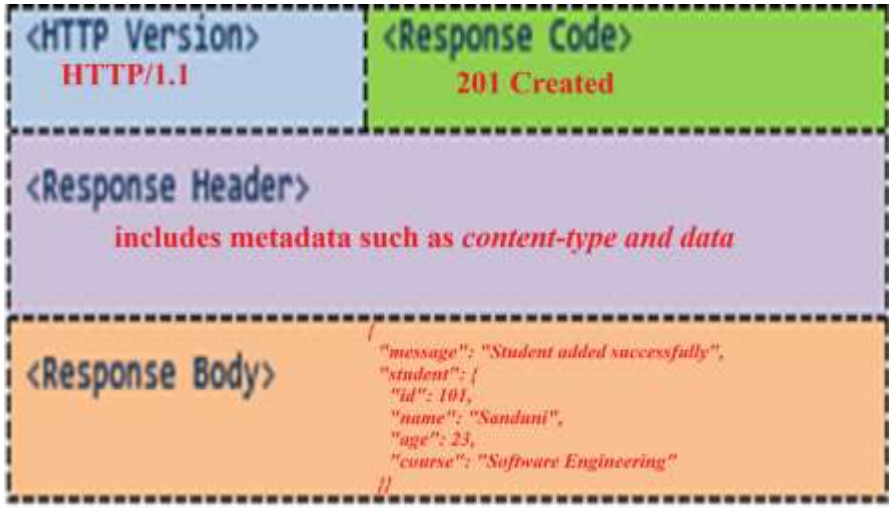
REST – HTTP Request and Response

Example:

Let's say you have a Student Management REST API running at: https: // api.school.com/students. Write API exposes endpoints to **Create, Read, Update** and **Delete** student records.

1. Create student





REST – HTTP Request and Response

2. Get all students

Request: GET https: //api.school.com/students

Response:

```
[
    { "id": 101, "name": “Sanduni", "age": 23 },
    { "id": 123, "name": “Perera", "age": 26 }
]
```

REST – HTTP Request and Response

3. Update All

Request: PUT https: //api.school.com/students/123

Body: { "name": "Kasun Perera", "age": 26 }

Response:

```
[
    "message": "Student updated successfully",
    "student": { "id": 123, "name": "Kasun Perera", "age": 26 }
]
```

REST – HTTP Request and Response

4. Update - Partial

Request: PATCH https: //api.school.com/students/123

Body: { "age": 27 }

Response:

```
[
    "message": "Student age updated successfully",
    "student": { "id": 123, "name": "Kasun Perera", "age": 27 }
]
```

REST – HTTP Request and Response

5. Delete Data

Request: DELETE https: //api.school.com/students/123

Response: { "message": "Student age updated successfully“ }

REST – HTTP codes

- HTTP status codes are **3-digit numbers** that indicate the status of a client's request to a server.
- The **first digit** defines the class of response.
  - 1xx Informational
  - 2xx Success
  - 3xx Redirection
  - 4xx Client Error
  - 5xx Server Error
- The **last two digits** provide specific details about the response.

REST – HTTP codes

Commonly used HTTP codes

- 200 OK (Success)
- 201 Created
- 202 Accepted
- 204 No content
- 301 Moved Permanently (Redirection)
- 400 Bad request
- 404 Not Found (Client Error)
- 500 Internal Server Error (Server Error)
- 502 Bad gateway
- 503 Service unavailable

Express JS

```
CONCEPTS (What you're building)

• 3-Tier Architecture  → WHERE things are deployed
• MVC Pattern          → HOW code is organized
• REST Principles      → HOW systems communicate

                       ↓

IMPLEMENTATION TOOL

• Express.js           → THE FRAMEWORK that
                         implements all the above
```

- A web application framework for Node.js
- Provides tools to build web servers and APIs
- Minimal and flexible
- Think of Express.js as
  - The *construction kit* you use to build your application
  - The *toolbox* that has everything you need
  - The *foundation* on which you implement 3-Tier, MVC, and REST.

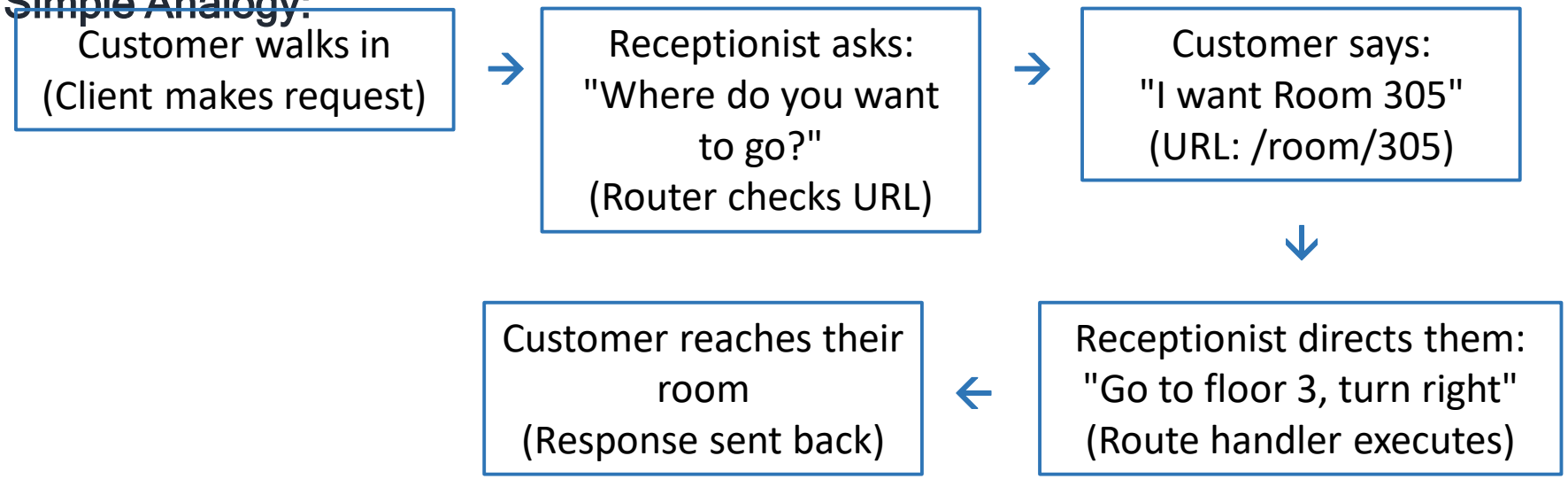
Express JS in 3-Tier and MVC

- Express.js operates in Tier 2 (Application Tier) and connects to both Tier 1 and Tier 3.
- It provides,
  - HTTP server functionality (listens for requests from Tier 1)
  - Middleware for processing requests
  - Database connection capabilities (connects to Tier 3)
  - Response handling (sends data back to Tier 1)
- Express.js provides the **building blocks** to implement MVC.
  - express.Router() → Maps to CONTROLLER
  - JavaScript Classes → Maps to MODEL
  - res.json() → Maps to VIEW

Express JS – Routing

Routing in Express.js is the mechanism that determines how an application responds to client requests for specific URLs (endpoints) and HTTP methods (GET, POST, PUT, DELETE).

Simple Analogy:



Express JS – Basic Routing Syntax

app.METHOD(PATH, HANDLER)

- app = Express application instance
- METHOD = HTTP method (get, post, put, delete, etc.)
- PATH = URL path (route)
- HANDLER = Function that executes when route is matched

Example:

```
app.get('/about', (req, res) => {
        res.send('This is the about page');
});
```

Express JS – Middleware

Middleware is a function that runs between the client request and the server response. It can modify the request, response, or terminate the request cycle.

- Express JS has **built-in middleware** functions for handling requests, parsing data, and serving static files.
- You can use **third-party middleware** functions for tasks like logging, authentication, and caching.
- You can create your **own (custom) middleware** functions to handle specific tasks or modify the request/response as needed.

Express JS – Working with database

- Express JS can work with different types of databases, including SQL databases (e.g., MySQL, PostgreSQL) and NoSQL databases (e.g., MongoDB).
- You can connect to a database using a database driver or an ORM (Object-Relational Mapping) library like Sequelize or Mongoose.
- Querying the database is done using SQL or a database-specific query language (e.g., MongoDB query syntax).
- ORM libraries provide a higher-level interface for working with databases and can simplify the code for common tasks like creating, reading, updating, and deleting records.

Express JS – REST APIs

- Express JS is well-suited for building RESTful APIs because of its routing and middleware capabilities.
- To create a RESTful API, you define routes that correspond to the different HTTP methods (GET, POST, PUT, DELETE) and the resources you want to expose.
- You can use middleware functions to handle tasks like input validation, authentication, and rate limiting.
- RESTful APIs should follow certain principles, such as using HTTP status codes to indicate success or failure, using meaningful URIs to identify resources, and providing a consistent interface for interacting with the API.

Tutorial – Let's build a simple Express JS backend

Case study: Student Management System

Create a backend API that allows you to:

- Add a new student
- Get all students
- Get a student by ID
- Update student info
- Delete a student

SLIIT
Discover Your Future

Thank You